

Relatório Técnico — Simulação dos Relógios Lógicos de Lamport

Implementação via gRPC entre 3 processos independentes (P1, P2, P3) em containers Docker

1. Cenário de Teste

A implementação segue a Abordagem A (sistemas distribuídos reais): cada processo (P1, P2, P3) roda em um contêiner Docker distinto, comunicando-se via gRPC. Um serviço adicional (Logger) coleta o log individual de cada processo via RPC e gera o log global ordenado pelo timestamp de Lamport, com desempate por ID de processo ($P1 < P2 < P3$).

O cenário executado dispara a mesma cadeia lógica $P1 \rightarrow P2 \rightarrow P3 \rightarrow P1 \rightarrow P3 \rightarrow P2 \rightarrow P1$, mas agora cada chamada gRPC é **não-bloqueante**: o processo chamador dispara a chamada com `stub.Metodo.future(msg)` (recurso nativo do stub síncrono do gRPC, sem precisar migrar para `grpc.aio`) e continua executando eventos internos (EXEC) localmente enquanto a resposta ainda não chegou, só bloqueando no `future.result()` quando o valor é de fato necessário. Além disso, o processo P2 executa 5 eventos internos *antes* de aceitar qualquer conexão, testando a regra de recebimento $L_j = \max(L_i, t_m) + 1$ com o relógio local já adiantado em relação ao timestamp recebido.

Cenário de teste: chamadas gRPC não-bloqueantes (.future()) entre P1 -> P2 -> P3 -> P1 -> P3 -> P2 -> P1
(cada processo executa EXECs locais enquanto aguarda a resposta pendente; P2 também 'adianta' 5 EXECs antes do 1o RECEIVE)

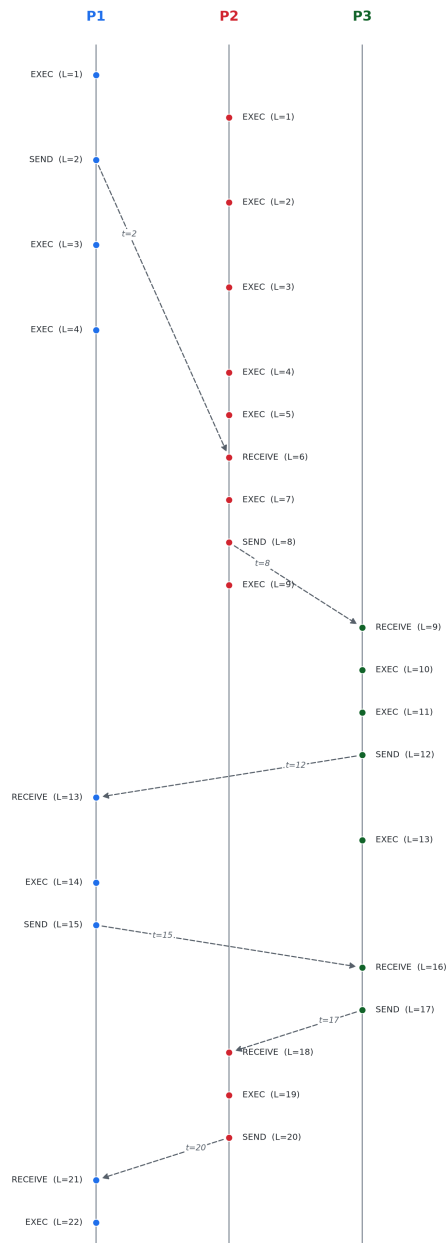


Figura 1 — Diagrama de sequência do cenário executado, com a evolução do relógio lógico (L) de cada processo e o timestamp (t) anexado a cada mensagem enviada. Note os trechos em que P1 e P2 avançam seus próprios EXECs em

paralelo, sem nenhuma mensagem entre eles nesse intervalo.

Um efeito interessante e não planejado surgiu dessa mudança: como P1 não bloqueia mais em `ExecuteTestCase` esperando a resposta de P2, quando P3 chama de volta `P1.ExecuteProcess` essa chamada é atendida por *outra* thread do pool gRPC de P1, rodando **concorrentemente** com a primeira — as duas compartilham o mesmo relógio lógico (`self.timestamp`). O `RECEIVE` final de P1 ($L=21 = \max(15,20)+1$) usa um valor de relógio que foi avançado pela outra chamada enquanto a primeira ainda esperava — uma concorrência real dentro do próprio processo, e não só entre processos distintos.

Trecho do log global gerado (arquivo `result/output.log`), destacando P1 e P2 avançando em paralelo logo após o primeiro `SEND`:

```
[Processo P1] Evento: SEND | Relógio Lógico: 2 | sent 'Process 1 sending message to process 2'
[Processo P2] Evento: EXEC | Relógio Lógico: 2 | Internal event
[Processo P1] Evento: EXEC | Relógio Lógico: 3 | Internal event
[Processo P2] Evento: EXEC | Relógio Lógico: 3 | Internal event
[Processo P1] Evento: EXEC | Relógio Lógico: 4 | Internal event
[Processo P2] Evento: EXEC | Relógio Lógico: 4 | Internal event
[Processo P2] Evento: EXEC | Relógio Lógico: 5 | Internal event
[Processo P2] Evento: RECEIVE | Relógio Lógico: 6 | received 'Process 1 sending message to process 2'
```

2. Análise das Limitações do Relógio de Lamport

O relógio de Lamport garante que, se um evento *a* aconteceu antes de um evento *b* ($a \rightarrow b$, por dependência causal direta ou por cadeia de mensagens), então necessariamente $L(a) < L(b)$. Porém a recíproca **não** é verdadeira: $L(a) < L(b)$ não implica que $a \rightarrow b$ — os dois eventos podem ser *concorrentes* (sem relação causal alguma), apenas tendo recebido estampas de tempo comparáveis por coincidência numérica.

Essa limitação aparece de forma ainda mais direta no log deste cenário: os eventos **P1 EXEC (L=3)** e **P2 EXEC (L=3)** têm exatamente o mesmo timestamp (o mesmo vale para $L=4$, em P1 EXEC e P2 EXEC). Nenhum dos dois foi causado pelo outro — P1 está apenas avançando localmente enquanto aguarda (de forma não-bloqueante) a resposta de P2, e P2 está executando seu próprio warm-up de 5 EXECs antes de sequer processar a mensagem de P1. São eventos genuinamente concorrentes que, por coincidência numérica, empataram no relógio de Lamport — exatamente o cenário em que o desempate por ID de processo ($P1 < P2 < P3$) do log global entra em ação, mas que **não** significa que um aconteceu antes do outro. Olhando só para o valor de *L* (ou mesmo para a ordem produzida pelo desempate), não há como saber que esses dois eventos são independentes — essa informação só existiria com um relógio vetorial.

Os **Relógios Vetoriais** resolvem exatamente esse problema: cada processo P_i mantém um vetor $V_i[1..N]$ (um contador por processo do sistema, $N=3$ neste trabalho). A cada evento interno ou envio, o processo incrementa apenas sua própria posição ($V_i[i]++$); ao receber uma mensagem com vetor V_m , atualiza $V_i[k] = \max(V_i[k], V_m[k])$ para todo *k*, e depois incrementa $V_i[i]$. Com isso, dados dois eventos *a* e *b* com vetores $V(a)$ e $V(b)$, é possível determinar exatamente a relação entre eles comparando componente a componente: $a \rightarrow b$ se e somente se $V(a) \leq V(b)$ em todas as posições (com pelo menos uma desigualdade estrita); se nem $V(a) \leq V(b)$ nem $V(b) \leq V(a)$, os eventos são comprovadamente concorrentes. Isso elimina o falso-positivo que o relógio escalar de Lamport pode gerar.

3. Dificuldades Encontradas e Lições Aprendidas

- **Sincronismo entre .proto e código gerado:** diversos bugs de execução (campo inexistente, `AttributeError`) vieram de editar o arquivo `.proto` sem regerar os arquivos `_pb2.py/_pb2_grpc.py` correspondentes — os dois precisam ser tratados como um par indissociável.
- **RPC sem retorno explícito:** um método de `servicer gRPC` que não executa `return` devolve `None` implicitamente, e o `gRPC` falha ao tentar serializar isso como resposta ('Failed to serialize response') — erro que só aparece em tempo de execução, não na inicialização do servidor.
- **Logging silencioso:** `logging.basicConfig()` sem o parâmetro `level` deixa o logger raiz em nível `WARNING` por padrão, descartando silenciosamente todas as chamadas `logger.info(...)` — os arquivos de log ficavam vazios sem nenhum erro visível, o que tornou o diagnóstico mais difícil.

- **Herança e instanciação incorreta:** ao extrair uma classe base `Process` compartilhada entre P1/P2/P3, um erro de instanciar a classe base em vez da subclasse especializada fez o servidor registrar um `servicer 'vazio'` — o bug não gerava exceção na inicialização, só se manifestava quando outro processo tentava efetivamente chamar o RPC.
- **Bind mount não recarrega processo em execução:** montar o código-fonte via volume no Docker Compose permite editar sem rebuildar a imagem, mas o container precisa ser recriado (`--force-recreate`) para que o interpretador Python releia o arquivo atualizado — simplesmente salvar o arquivo não era suficiente.
- **Ordem de inicialização entre serviços independentes:** como P1, P2, P3 e o `Logger` sobem em containers separados e se conectam entre si, foi necessário adicionar `healthchecks` e `depends_on: condition: service_healthy` no Docker Compose para garantir que um processo só tente chamar outro depois que o gRPC do destino estiver de fato aceitando conexões — apenas o container estar 'Up' não é garantia suficiente.
- **Condição de corrida ao introduzir chamadas não-bloqueantes:** ao trocar as chamadas síncronas por `stub.Metodo.future(...)`, um mesmo processo passou a atender duas RPCs concorrentemente em threads diferentes do pool gRPC, ambas incrementando o mesmo contador `self.timestamp` sem nenhum lock. O resultado observado ficou consistente nos testes realizados (o GIL do Python serializa a operação de incremento na prática, nessa escala), mas não há garantia formal de atomicidade — uma implementação mais rigorosa precisaria de um `threading.Lock()` ao redor do acesso ao relógio lógico.

A principal lição do trabalho foi perceber que, em um sistema distribuído real (mesmo pequeno, com 3 processos), a maior parte dos bugs não está na lógica do algoritmo em si (os incrementos e o `max()` de Lamport são triviais de implementar corretamente), e sim na infraestrutura de comunicação e inicialização entre processos independentes — exatamente o tipo de dificuldade que a ementa da disciplina busca expor na prática.